



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

基于热电芯片的实时天气监测系统的研究

学院 电子信息与电气工程学院、生物医学工程学院

组别 一组

学号 522031910790、523031910352

523031910353、523031910249

人员 杜云天、方兴宇、李东睿、张懿

指导老师 蔡兴汉、胡志宇

2025 年 1 月 6 日

摘 要

本文研究了基于热电芯片的太阳辐射强度实时测量系统。通过分析现有云量观测技术的不足，提出了一种利用热电芯片实现光-热-电能量转化的新方法。

在室内实验中，使用氙光手电筒照射电热芯片，测量其相应信号，证实电热芯片的响应与光照强度呈现良好的线性关系，证明使用电热芯片观测天气具有极强的可行性。

在室外实验中，从韦伯望远镜中借鉴了灵感，采用六边形镜子改进太阳灶的聚光能力。同时，在电热芯片表面涂抹颜料，分析其光谱响应特性。结果表明，热电芯片能够高效捕获太阳辐射，对光谱成分具有显著的响应能力。通过针对太阳光进行直接测量以及使用摄像头等手段对天空进行探测，并使用图像处理技术，进一步验证了该系统在晴天分辨和光谱成分分析中的可行性。

该研究为气象观测、太阳能发电和环境研究提供了一种低成本、高灵敏度的解决方案。并指出使用电热芯片探测天气面临的一系列挑战以及未来可行的实验装置改进方案与实验进行方案。

关键词：热电芯片，太阳辐射，光谱响应，晴天分辨，聚光装置，气象观测

1 项目背景与目的

1.1 已有天气测量系统

现有的云量观测方式已从传统的人工观测发展到多种现代技术手段。

传统方法包括人工肉眼观测，观测员通过目视估算云层覆盖比例，方法简单实用但具有较强的主观性。

随着科技进步，现代气象卫星遥感技术观测云量成为主流，地球同步卫星和极轨卫星配备的多光谱仪器，可定量测量云的覆盖范围、类型、厚度和高度；激光雷达卫星如 CALIPSO 则通过垂直剖面数据详细监测高云和低云。地基观测手段也日益完善，云雷达采用毫米波探测云层结构，全景摄像系统结合图像处理和 AI 算法自动分析云量，激光测云仪通过激光反射测算云层高度和厚度。此外，卫星观测与地面设备的结合、无人机携带成像与探测设备的应用，进一步提升了云量监测的精确性和覆盖范围。现代技术手段让云量观测更加科学、高效，为天气预报、航空航天、农业气象和环境监测等领域提供了重要支持。

1.2 项目意义

现有的观测设备成本高，操作难度大，且直接观测云量难以直观获得地面阳光相关的数据信息。因此，我们希望能给出一种较为直观的探测方式，清晰准确地获取地面太阳辐照强度的方式，并将其用于指导以下生产活动。

- **气象与气候研究：**随着全球气候变化日益加剧，精准的天气监测是研究气候变化、预测天气和推动环境治理的基础。电热芯片因其高灵敏度和高观测带宽的优势使其非常适合用来检测太阳辐射热相关的指标。
- **太阳能发电：**在新能源和碳中和背景下，实时监测太阳辐射是太阳能发电规划、分布和效能提升的关键。
- **农业生产：**天气变化的实时监测能帮助农民优化生产安排，提升产量和质量，推动农业现代化。
- **日常生活：**天气监测与预报对日常活动的指导作用至关重要，帮助人们合理规划户外活动等。

1.3 项目优势

相较于光电型天空监测装置，电热芯片波长覆盖范围更广。电热芯片直接将各个波长的光转化为热，测量光的热效应，覆盖光的波长范围更大，测量更为精准。

对比卫星和遥感技术，电热芯片无需外部电源和通信支持，功耗更低，且对环境适应性更强。电热芯片直接将热量转化为电信号，无需外部能源驱动，测量设施较为简单，

成本相较于卫星技术成本极低。电热芯片能在各种复杂环境中工作，对环境的适应能力较强。

热的监测更符合人体对天气变化的直观感受。电热芯片将光转化为热，与人体皮肤对太阳光的感受模式相同，能更为直接模仿人体对天气的感受方式，参考意义较强

现有热电式天气监测系统缺乏对不同天气条件下辐射组成变化的分析。在测量中，我们使用不同颜色的马克笔涂抹在电热芯片的表面上，旨在分析不同颜色的吸光情况，从而间接反应太阳光中不同波长颜色的光的热效应的区别。

本研究旨在通过直接测量热量，实时、精准地评估晴天程度。利用材料的吸收与反射特性差异，以及热电芯片的光-热-电转化机制，分析不同晴天程度下太阳辐射的成分、强度、能量与热电芯片电压之间的关系。

2 实验原理与装置

2.1 实验原理

2.1.1 塞贝克效应

塞贝克效应描述了由两种不同导体（或半导体）组成的闭合回路中，当两接点温度不同 (T_1, T_2) 时，回路中会产生电流，形成热电回路。回路中的电动势称为塞贝克电动势。[1]

在芯片的热端和冷端温度分别为 T_h 和 T_c 时，P 型和 N 型热电材料的接触点会产生电压差。该电压差与温度差成正比，通过测量电压差可求得 $T_h - T_c$ 。

2.1.2 珀尔帖效应

珀尔帖效应描述了在不同导体间的结点处，当有电流流过时，会发生吸热或放热的现象。[2]

如图1所示，当电流从 P 型材料流向 N 型材料时，界面处吸收热量，导致冷却。热量随后通过 P 型和 N 型材料传递至热端，并在散热器处释放。

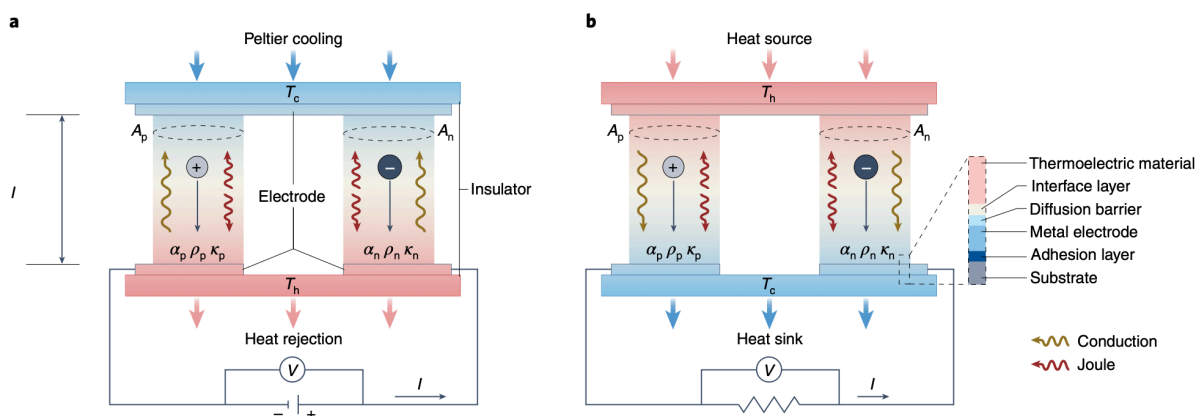


图 1: 珀尔帖效应与塞贝克效应示意图 [3]

2.2 实验基础芯片

实验中选用热电芯片 TEC1-12715，其热端在受照射升温时会产生电压信号。通过测量输出电压，可推断太阳辐射的强弱。为增强弱光条件下的信号强度，实验设计了聚光装置，并使用覆盖不同颜色的材料分析光谱响应特性。

3 实验设计与改善

3.1 聚光装置

由于热电芯片在非直射光下输出电压较弱，我们设计了聚光装置以增强热端光功率。

初期选用太阳能灶进行聚光实验，但因太阳能灶表面平整度不足，其反射效果不理想，难以汇聚天空发出的散射光。为优化装置，我们参考韦伯望远镜的主镜设计，铺设 18 块六边形平面镜，实现更加平整和可控的反射效果。改进后的装置如图6和7所示。

通过手持激光笔验证，改进后的装置能够将平行光汇聚至抛物面焦点附近，满足实验要求（图4）。

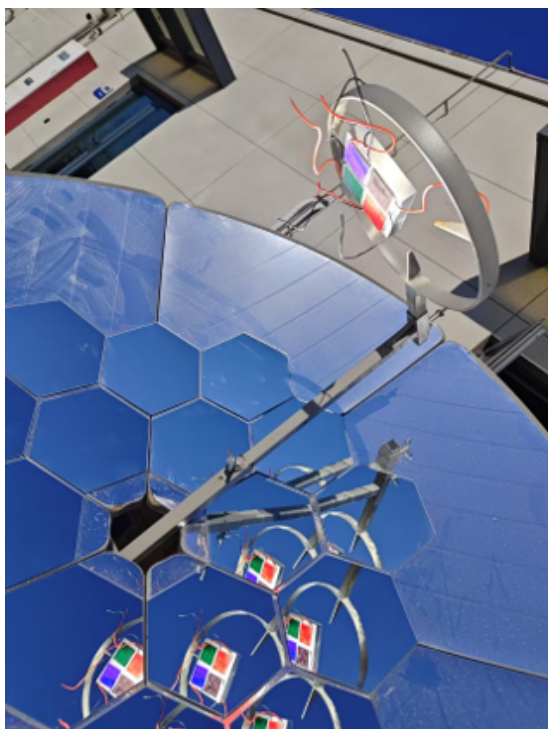


图 2: 聚光装置及热电芯片布置



图 3: 平面镜铺设设计

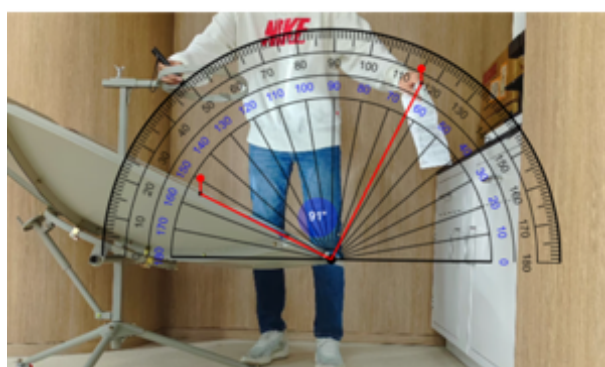


图 4: 利用激光验证验证焦点位置

实验测得太阳灶参数为直径 135cm、深度 23cm，计算得焦距为 50cm。利用 Python 进行光路仿真，结果如图所示：

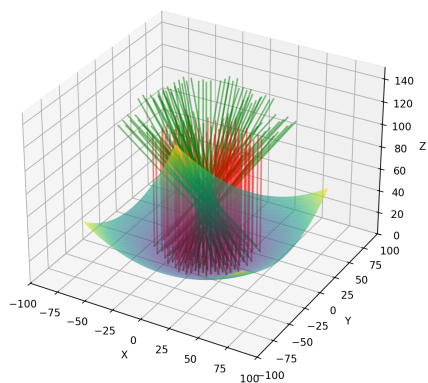


图 5: 光路仿真结果

以上调整显著提高了聚光装置的性能，使得热电芯片阵列能够更高效捕获太阳辐射。

3.2 热电芯片阵列

一个典型的热电芯片由热电偶组成，这些热电偶由交替排列的 P 型和 N 型腿构成。在实验之初，我们使用单块电热芯片探究不同方向上的云量对电热芯片示数的影响，发现在光照下，电热芯片的示数会产生一定的变化，但效果与云量的相关性不甚明显。针对这一问题，我们采用电热芯片阵列的模式改进了原有的测量方式。

在前期，我们使用彩纸黏贴在电热芯片上，后发现彩纸吸热后可能无法将能量全部传递至电热芯片表面，故后期改用马克笔涂抹的方式改变电热芯片表面颜色。



图 6: 改机前探测装置设计

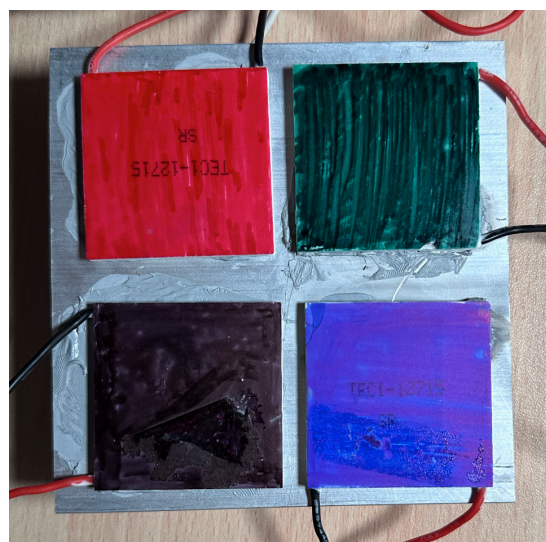


图 7: 改进后探测装置设计

4 实验结果与分析

4.1 芯片特性

为全面评估芯片的性能，我们在严格控制的实验室条件下对芯片进行了特性标定。实验设计主要涵盖以下三个方面：调整光源与热电芯片之间的距离、改变热电芯片表面覆盖纸张的颜色，以及采用不同颜色的马克笔对芯片表面进行涂色。

4.1.1 光强与电压的关系

实验中使用 40W 氙灯手电筒作为光源，通过台式万用表和手持式照度计分别测量输出电压和光照强度。

通过调节光源与芯片热端的距离，记录光照强度、电压及距离数据，实验结果如图16和图17所示：

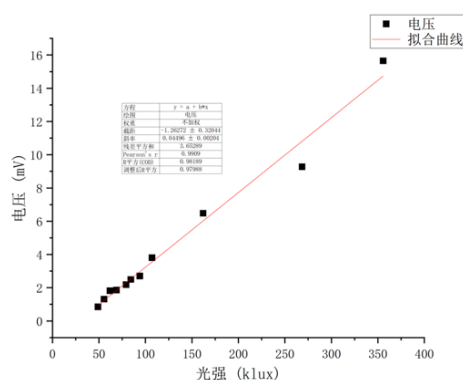


图 8: 电压与光照强度的关系

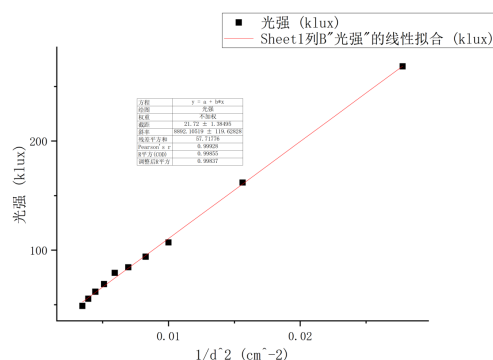


图 9: 光照强度与距离的关系

从图16可以看出，在室内光源条件下，芯片输出电压与光照强度呈现良好的线性关系。同时，图17表明光照强度与光源距离平方呈反比关系。这表明热电芯片能够响应光强变化，并具备辨别光强的能力。

4.1.2 颜色与电压的关系

为了进一步探讨热电芯片对光谱成分的响应能力，我们通过改变芯片表面接收到的光色来分析其电压输出特性。具体方法包括在热电芯片表面覆盖不同颜色的纸张，以及使用不同颜色的马克笔对芯片表面进行涂色，以改变光谱成分的吸收特性。

首先，在无任何覆盖或涂色的情况下，固定光源与热电芯片之间的距离，测量热电芯片在光照条件下的电压输出。实验结果如图10所示。

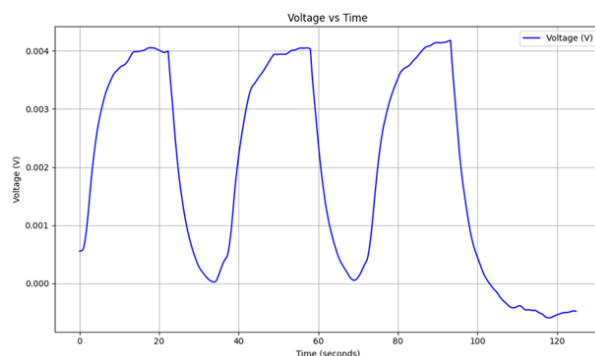


图 10: 热电芯片在无覆盖条件下的电压响应

随后，采用红色、绿色和蓝色三种纸张分别覆盖在热电芯片表面，保持光源与芯片的距离不变，测量其电压输出。实验结果如图15所示。从图中可以看出，热电芯片的电压值随着覆盖颜色的不同呈现显著差异，其中绿色覆盖条件下的电压值最高，其次是红色，蓝色最低。这表明热电芯片的输出特性受到覆盖材料光谱吸收特性的显著影响。

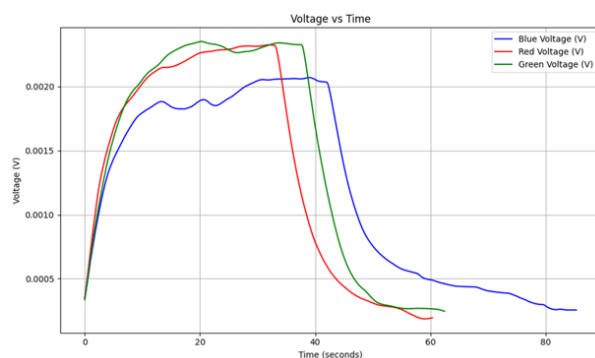


图 11: 不同颜色纸张覆盖下的电压响应

为探究热电芯片的吸收特性，我们采用黑色、蓝色、红色和绿色四种马克笔分别对热电芯片表面进行涂色，保持光源与芯片的距离不变，测量其电压响应特性。实验结果如图12所示。实验发现，黑色涂色条件下的电压值最高，而绿色、红色和蓝色涂色的电压值依次递减，与覆盖材料的光谱选择性吸收特性一致。

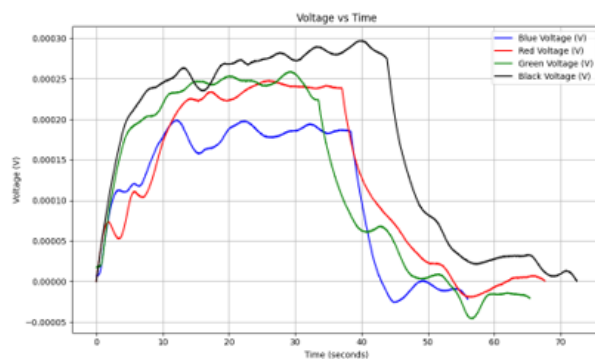


图 12: 不同颜色涂色条件下的电压响应

以上实验结果表明，热电芯片对光谱成分具有显著的响应能力，其电压输出特性受到覆盖材料或涂色材料光谱吸收特性的影响。

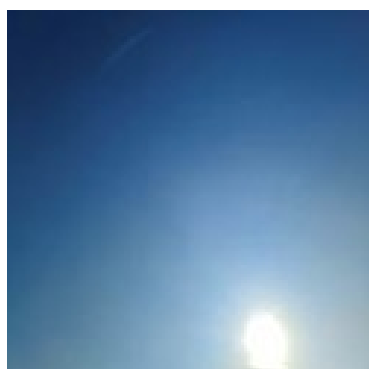
4.2 晴天分辨能力

我们在开阔室外进行测试，将热电芯片进行红绿蓝三颜色纸张覆盖，使用手持式电压计来记录热电芯片的电压值。记录聚光装置朝向方向的图片。

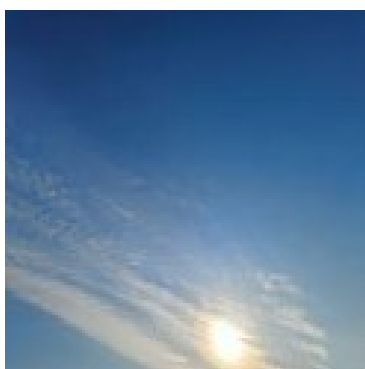
4.2.1 天空颜色分析

拍摄聚光装置朝向方向的图片，将图片调整为 200×200 像素，得到图片像素的 rgb 值密度分布 [3]。

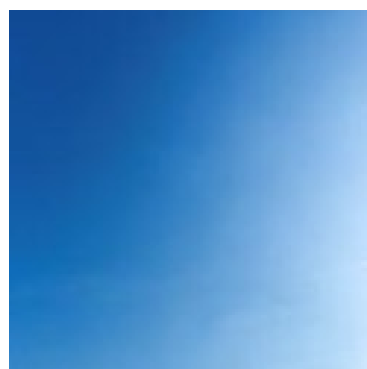
为了更直观地展示实验结果，以下为六组实验数据的对比图：



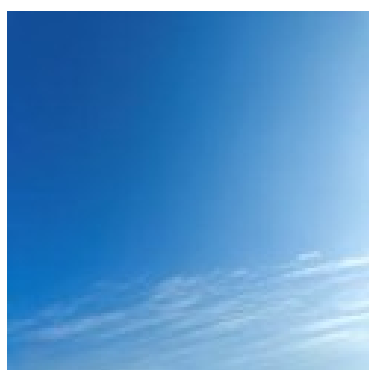
(a) 图 1



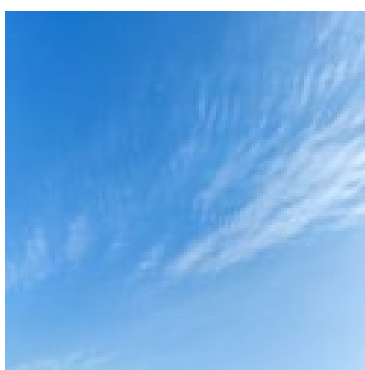
(b) 图 2



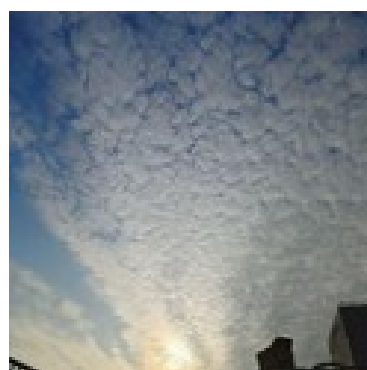
(c) 图 3



(d) 图 4



(e) 图 5



(f) 图 6

图 13: 六组天空对比图

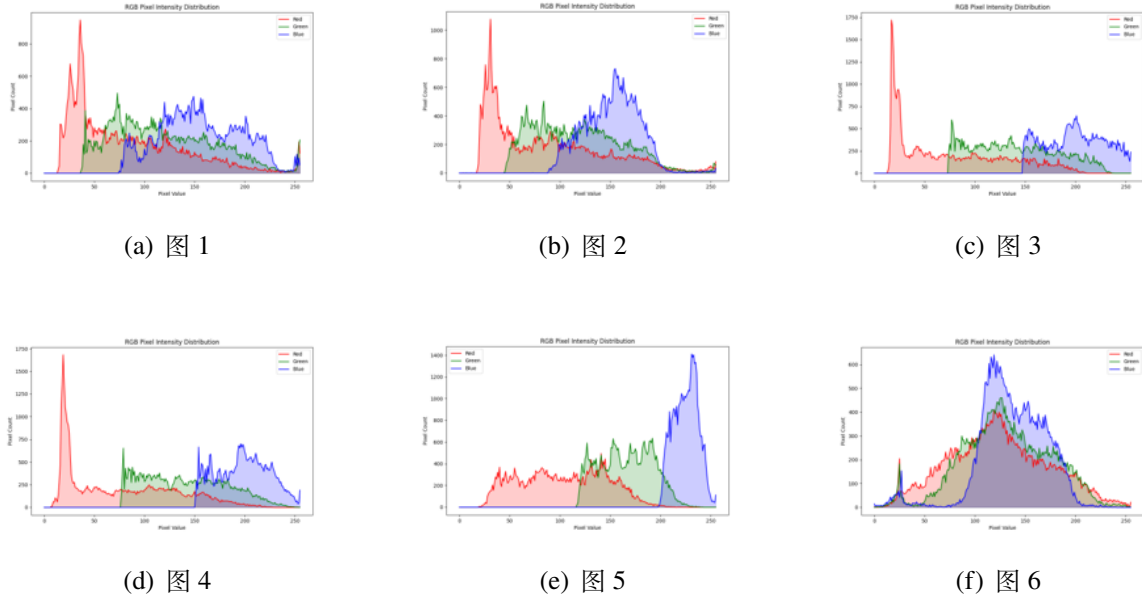


图 14: 六组天空 rgb 值密度分布

电压值在聚光装置不同朝向下展现出显著差异，归因于晴天程度差异下云层，光的色温、强度等差异。在实验环境下，与红色覆盖相比，绿色与蓝色覆盖展示出更高的电压，阳光中的成分更易被绿、蓝覆盖吸收。不同颜色覆盖的电压排布顺序差异，表明不同天气条件下辐射成分的差异可被探测。

4.2.2 天空热量分析

我们将电热芯片直接放置在地面上，朝向不同的方向，记录黏贴有不同颜色纸张的电热芯片的示数，并记录所朝向方向的光照强度，分析不同电热芯片的示数与光照强度之间的关系，结果如下图所示 [4]。



图 15: 太阳光直射时不同颜色纸张覆盖下的电压响应

电压值在电热芯片块不同朝向下展现出显著差异，归因于晴天程度差异下云层，光的色温、强度等差异。在实验环境下，与红色覆盖相比，绿色与蓝色覆盖展示出更高的

电压，阳光中的成分更易被绿、蓝覆盖吸收。能量与电压的拟合结果呈现一定线性，验证了装置对晴天程度差异下太阳辐射能量的探测能力。

而后，我们尝试将电热芯片固定在聚光镜交点处，尝试将某一方向上天空发出的光线全部汇聚在电热芯片上，结果发现除非朝向太阳，电热芯片的示数始终为负数。分析之后发现是因为太阳发出的光比其他方向天空的光强约 70 倍，电热芯片背面的散热铝块被加热之后导致示数翻转。我们尝试使用纸箱遮挡的方式排除这一干扰因素，结果以失败告终，纸箱仍然会吸收热量并加热电热芯片反面。

由于室外环境复杂，干扰难以完全排除，但这个实验依旧反映了使用电热芯片检测天气的可能性，但装置细节仍然需要做出一定的改进。

4.2.3 太阳能量分析

人对晴天的感受很大程度上取决于阳光的强度，而阳光的强度很大程度上取决于当前的太阳高度角以及太阳被云遮挡的情况。因此，我们决定直接将电热芯片对准太阳，测量太阳的热量在电热芯片上的反应，结果如图所示，从左到右分别对应于蓝色、黑色、红色、绿色。

实验证实，电热芯片组能有效测量太阳的热效应，且电压示数的高低为黑色大于蓝色大于绿色大于红色，与理论预测一致。该实验证明使用电热芯片探测晴天状况具有较高可行性。



图 16: 电热芯片直接受太阳照射时示数



图 17: 太阳灶聚焦后电热芯片示数

5 项目总结与展望

5.1 项目总结

在项目中，我们对热电芯片在光-热-电转换中的特性进行了深入研究。通过购买氙灯手电筒以及照度仪，测量电热芯片在不同光强下的信号强度，绘制了电热芯片输出电压与光照强度的关系。

而后，我们采集了太阳光并对太阳光进行聚焦，研究在阳光照射下，不同颜色芯片相应信号的情况。我们使用太阳灶构建出一个基于热电芯片阵列的晴天程度实时分辨系统。并使用摄像头拍摄天空，分析该方向上的云量情况，编写 Python 程序，计算该方向上 RGB 像素点的分布情况。

该系统展现出对于不同晴天程度的天空的分辨本领，并能够为太阳辐射的成分与强度分析提供一定的参考。

相较于之前的研究，我们从能量角度展开了更为细致的分析，并在颜色、电压、能量之间构建出关系，为项目提供理论支持。

5.2 项目缺憾

项目实施过程中仍然有许多缺陷，可以留给以后的学生加以完善。

第一，电热芯片的冷端冷却与信号稳定性问题。在项目实施的过程中，我们首先尝试将电热芯片安装在太阳灶的焦点处，测量电热芯片的电压信号，发现电热芯片的输出始终为负值，这意味着电热芯片的受光面的温度低于冷端，与理论预测不符合。分析原因，我们发现是因为电热芯片的冷端被太阳光加热导致升温，由于太阳光光强很大，致使铝块升温速度事实上快于受光照端。我们采用纸盒遮挡等方法均无法有效解决这一问题。建议可以采用在冷端放置冰水等方式尽量将冷端维持在一个较为恒定的温度。

第二，电热芯片的固定安装问题。电热芯片目前直接通过铝块固定在支架上面，需要手动摇动锅的位置，将光线的焦点对准在电热芯片上。这会导致每次转动锅时都需要目视调整光线聚焦的位置，操作不方便，且受到每次操作的干扰较大。建议尝试在锅上连接支架，确保电热芯片组始终处在锅的焦点的位置，这样每次对准一块天空时就不需要反复调整锅的角度和电热芯片的位置，能大大简化操作。

第三，电热芯片的实时测定问题。电热芯片测量依赖于手持式万用表，且收到风与阳光照射的影响较大。如果要同时采集多路数据，只能通过录制视频并手动输入电脑的方式，对数据处理、寻找不同颜色电热芯片之间数据的联系造成了困难。需要改进测量仪器，使用多通道的 AD 转化卡来完成操作。

5.3 项目展望

在一学期的实验过程中，我们成功使用了电热芯片探测了不同方向云量对电热芯片进行照射所产生的信号，证实了使用电热芯片实时监测天气状况这一想法是具有一定可

行性的。但是，碍于技术不足及时间有限等困难，项目并非十分完善，仍然有较大的改进空间。下面，我将抛砖引玉，尝试为后来者提供一些思路以及启示。

电热芯片对温度的变化非常敏感，而通过手持式光度计测量向阳面与背阳面的光照强度，会发现差距来到惊人的 70 倍，这意味着单单进行挡光操作并不能保证电热芯片的恒温，还需要进行特别的隔热处理。

电热芯片对风非常敏感，在室外实验时，如果刮风，电热芯片的电压示数就会发生非常明显的漂移，建议可以尝试在一个全玻璃的房间内完全隔绝外界天气的干扰进行实验。

6 代码示例

示例 1：计算反射光线角度及绘图代码

```
1 import random
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5
6 def find_nearest_point(points, input_point):
7     """
8     在一系列点中找到离输入点最近的一个点。
9     :param points: 点的集合，格式为 [(x1, y1), (x2, y2), ...]
10    :param input_point: 输入点，格式为 (x, y)
11    :return: 离输入点最近的点和距离
12    """
13    nearest_point = None
14    min_distance = float('inf')
15
16    for point in points:
17        distance = ((point[0] - input_point[0])**2 + (point[1] -
18            input_point[1])**2)**0.5
19        if distance < min_distance:
20            min_distance = distance
21            nearest_point = point
22
23    return nearest_point, min_distance
24
25 points = [(10*np.sqrt(3)*np.cos(np.pi/6), 10*np.sqrt(3)*np.sin(np.pi/6)),
26           (10 * np.sqrt(3) * np.cos(np.pi / 2), 10 * np.sqrt(3) * np.sin(np
27             .pi / 2)),
28           (10*np.sqrt(3)*np.cos(5*np.pi/6), 10*np.sqrt(3)*np.sin(5*np.pi/6)
29             ),
```

```

28         (10*np.sqrt(3)*np.cos(7*np.pi/6), 10*np.sqrt(3)*np.sin(7*np.pi/6)
29         ),
30         (10*np.sqrt(3)*np.cos(-np.pi/2), 10*np.sqrt(3)*np.sin(-np.pi/2)),
31         (10*np.sqrt(3)*np.cos(-np.pi/6), 10*np.sqrt(3)*np.sin(-np.pi/6)),
32         (30, 0),
33         (30 * np.cos(1 * np.pi / 3), 30 * np.sin(1 * np.pi / 3)),
34         (30 * np.cos(2 * np.pi / 3), 30 * np.sin(2 * np.pi / 3)),
35         (30 * np.cos(3 * np.pi / 3), 30 * np.sin(3 * np.pi / 3)),
36         (30 * np.cos(4 * np.pi / 3), 30 * np.sin(4 * np.pi / 3)),
37         (30 * np.cos(5 * np.pi / 3), 30 * np.sin(5 * np.pi / 3)),
38         (20 * np.sqrt(3) * np.cos(0 * np.pi / 3 + np.pi / 6), 20 * np.
39         sqrt(3) * np.sin(0 * np.pi / 3 + np.pi / 6)),
40         (20 * np.sqrt(3) * np.cos(1 * np.pi / 3 + np.pi / 6), 20 * np.
41         sqrt(3) * np.sin(1 * np.pi / 3 + np.pi / 6)),
42         (20 * np.sqrt(3) * np.cos(2 * np.pi / 3 + np.pi / 6), 20 * np.
43         sqrt(3) * np.sin(2 * np.pi / 3 + np.pi / 6)),
44         (20 * np.sqrt(3) * np.cos(3 * np.pi / 3 + np.pi / 6), 20 * np.
45         sqrt(3) * np.sin(3 * np.pi / 3 + np.pi / 6)),
46         (20 * np.sqrt(3) * np.cos(4 * np.pi / 3 + np.pi / 6), 20 * np.
47         sqrt(3) * np.sin(4 * np.pi / 3 + np.pi / 6)),
48         (20 * np.sqrt(3) * np.cos(5 * np.pi / 3 + np.pi / 6), 20 * np.
49         sqrt(3) * np.sin(5 * np.pi / 3 + np.pi / 6))
50     ]
51
52 # 定义抛物面的焦距
53 f = (135/2)**2/(4*23) # 焦距
54 print(f)
55
56 # 生成抛物面数据
57 x = np.linspace(-70, 70, 200)
58 y = np.linspace(-70, 70, 200)
59 X, Y = np.meshgrid(x, y)
60 Z = (X ** 2 + Y ** 2) / (4 * f)
61
62 # 定义入射光线的方向（假设是平行光）
63 incident_direction = np.array([0, 0, -1]) # 入射光线沿着z轴负方向
64
65 # 计算某点的法向量
66 def normal_vector(x, y, f):
67     # 法向量的方向是由抛物面梯度给出的
68     # 对抛物面方程  $Z = (x^2 + y^2) / (4f)$  求梯度
69     nx = x / (2 * f)
70     ny = y / (2 * f)

```

```

66     nz = -1
67     t = np.array([nx, ny, nz])
68     return t / np.linalg.norm(t)
69
70
71 # 生成一系列入射光线的位置
72 x_positions = np.linspace(-100, 100, 30)
73 y_positions = np.linspace(-100, 100, 30)
74
75 # 创建图形
76 fig = plt.figure()
77 ax = fig.add_subplot(111, projection='3d')
78
79 # 绘制抛物面
80 ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.6)
81
82 # 绘制入射光线和反射光线
83 for x0 in x_positions:
84     for y0 in y_positions:
85         # 计算该点的z坐标（抛物面上的点）
86         z0 = (x0 ** 2 + y0 ** 2) / (4 * f)
87         point_on_paraboloid = np.array([x0, y0, z0])
88
89         # 计算该点的法向量
90         near_point, distance = find_nearest_point(points, (x0, y0))
91         if distance >= 10:
92             continue
93         normal = normal_vector(near_point[0], near_point[1], f)
94
95         # 计算反射光线的方向
96         dot_product = np.dot(incident_direction, normal)
97         reflection_direction = incident_direction - 2 * dot_product *
            normal
98
99 # 绘制入射光线
100 m = -100
101 # 绘制延长后的反射光线
102 in_end = point_on_paraboloid + m * incident_direction
103 ax.plot([point_on_paraboloid[0], in_end[0]],
104         [point_on_paraboloid[1], in_end[1]],
105         [point_on_paraboloid[2], in_end[2]], color='r', alpha=0.5)
106 # ax.quiver(point_on_paraboloid[0], point_on_paraboloid[1],
107            point_on_paraboloid[2],
108            incident_direction[0]*m, incident_direction[1]*m,
109            incident_direction[2]*m, color='r', alpha=0.5)

```

```

108
109     # 更改延长倍数，确保光线的延长能看到
110     k = 150 # 更大的延长倍数
111     reflection_end = point_on_paraboloid + k * reflection_direction #
        延长反射光线
112     # reflection_direction = k * reflection_direction
113
114     # 绘制延长后的反射光线
115     ax.plot([point_on_paraboloid[0], reflection_end[0]],
116             [point_on_paraboloid[1], reflection_end[1]],
117             [point_on_paraboloid[2], reflection_end[2]], color='g',
                alpha=0.5)
118
119     # 设置坐标轴标签
120     ax.set_xlabel('X')
121     ax.set_ylabel('Y')
122     ax.set_zlabel('Z')
123
124     # 设置图形的显示范围
125     ax.set_xlim([-100, 100])
126     ax.set_ylim([-100, 100])
127     ax.set_zlim([0, 150])
128
129     # 显示图形
130     plt.show()

```

示例 2：天空云量简单分类模型

```

1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  def optimized_adaptive_threshold(image, block_size):
7      # 创建一个均值滤波核
8      kernel = np.ones((block_size, block_size), dtype=np.float32) / (
        block_size * block_size)
9
10     # 使用卷积计算每个像素的邻域均值
11     mean_val = cv2.filter2D(image.astype(np.float32), -1, kernel)
12
13     # 设置常数 C，并生成二值图像
14     C = np.where(mean_val > 200, 30,
15                 np.where(mean_val > 150, 10,
16                     np.where(mean_val > 100, 0,
17                         np.where(mean_val > 50, -30, -70))))

```

```

18
19 # 计算阈值化结果
20 thresh = (image > (mean_val - C)).astype(np.uint8) * 255
21
22 return thresh
23
24
25 # 读取图片
26 image = cv2.imread('./data/cloud16.jpg')
27
28 if image is None:
29     print("Error: Image not found.")
30 else:
31     # 将BGR图像转换为HSV图像
32     hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
33
34     # 提高蓝色的饱和度和降低亮度
35     mask = (hsv_image[:, :, 0] >= 100) & (hsv_image[:, :, 0] <= 140) & (
36         hsv_image[:, :, 1] >= 50)
37     hsv_image[:, :, 1][mask] = np.clip(hsv_image[:, :, 1][mask] * 1.1, 0,
38         255)
39     hsv_image[:, :, 2][mask] = np.clip(hsv_image[:, :, 2][mask] * 0.85, 0,
40         255)
41
42     # 将HSV图像转换回BGR图像
43     image = cv2.cvtColor(hsv_image, cv2.COLOR_HSV2BGR)
44
45     # 转换为灰度图
46     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
47
48     # 使用优化后的自适应阈值
49     block_size = int((image.shape[0] + image.shape[1]) / 50) # 邻域大小
50     thresh = optimized_adaptive_threshold(gray_image, block_size)
51
52     # 计算云的面积
53     cloud_area = np.sum(thresh == 255)
54     total_area = thresh.shape[0] * thresh.shape[1]
55     cloud_fraction = cloud_area / total_area
56     print(f"Cloud fraction: {cloud_fraction*100:.2f}%")
57
58     # 显示原始图片和云检测结果
59     plt.figure(figsize=(10, 5))
60
61     plt.subplot(1, 2, 1)
62     plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

```

```

60     plt.title('Original_Image')
61
62     plt.subplot(1, 2, 2)
63     plt.imshow(thresh, cmap='gray')
64     plt.title('Cloud_Detection')
65
66     plt.text(0.5, 0.5, f'Cloud_Fraction: {cloud_fraction*100:.2f}%',
67             ha='center', va='center', color='red', fontsize=12, bbox=dict(
68                 facecolor='white', alpha=0.5))
69
70     plt.savefig('cloud_16.png')
71
72     plt.show()

```

示例 3: 天空 RGB 像素点分析模型

```

1  # from PIL import Image
2
3  # 打开图片文件
4  # img = Image.open('./data/cloud13.jpg')
5
6  # 将图片重塑为200x200大小
7  # Image.ANTIALIAS是一个过时的选项，现在可以使用Image.Resampling.LANCZOS
8  # resized_img = img.resize((200, 200), Image.Resampling.LANCZOS)
9
10 # 保存或显示图片
11 # resized_img.save('resized_image13.jpg')
12 # resized_img.show()
13
14 import cv2
15 import numpy as np
16 import matplotlib.pyplot as plt
17
18 # 读取图片
19 img = cv2.imread('resized_image21.jpg')
20
21 if img is None:
22     print("Error: Image not found.")
23 else:
24     # 将图片从BGR转换为RGB
25     img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
26
27     # 初始化R、G、B的计数器
28     r_counts = np.zeros(256, dtype=int)
29     g_counts = np.zeros(256, dtype=int)
30     b_counts = np.zeros(256, dtype=int)
31

```

```

32     # 统计每个R、G、B值的出现次数
33     for row in img_rgb:
34         for pixel in row:
35             r, g, b = pixel
36             r_counts[r] += 1
37             g_counts[g] += 1
38             b_counts[b] += 1
39
40     # 绘制折线图并填充下方面积
41     plt.figure(figsize=(10, 6))
42     plt.plot(r_counts, color='red', label='Red')
43     plt.plot(g_counts, color='green', label='Green')
44     plt.plot(b_counts, color='blue', label='Blue')
45     plt.fill_between(range(256), r_counts, color='red', alpha=0.2)
46     plt.fill_between(range(256), g_counts, color='green', alpha=0.2)
47     plt.fill_between(range(256), b_counts, color='blue', alpha=0.2)
48     plt.xlabel('Pixel_Value')
49     plt.ylabel('Pixel_Count')
50     plt.title('RGB_Pixel_Intensity_Distribution')
51     plt.legend()
52     plt.savefig('pixel_intensity_distribution21.png')
53     plt.show()

```

参考文献

- [1] 曾辉、曾琪. 塞贝克效应及其应用. pages 52–54, 2004.
- [2] 陈文娟, 薛德宽, 黄兴洲, 位浩杰, and 刘冰. 珀尔帖效应与塞贝克效应综合演示仪. In 大学物理实验, volume 28, pages 81–84. 中国石油大学 (华东), 2015.
- [3] Qihao Zhang, Kangfa Deng, Lennart Wilkens, Heiko Reith, and Kornelius Nielsch. Micro-thermoelectric devices. In *Nature Electronics*, volume 5, pages 333–347, 2022.
- [4] Zekun Liu, Shuai Zhang, Zhenhua Wu, Xing-Er Wang, Kangning Zou, Zhiyuan Zhu, Xuyu Wang, Erzhen Mu, Xiaotian Zhang, Yan Liu, Huilie Shi, and Zhiyu Hu. Visual-audio thermoelectric detectors for images and sound recognition. In *Advanced Functional Materials*, volume 34, page 2406110, 2024.